

SmolLM2 Zybo 5day DMA Agent Guide v4

- [SmolLM2 Zybo 5일 DMA/Agent 선형 실행 가이드 v4](#)
 - [0. 지금 프로젝트 판정](#)
 - [1. 최종 하드웨어/소프트웨어 흐름](#)
 - [2. Codex/AI agent 권한과 안전 규칙](#)
 - [3. 선형 실행표](#)
- [S00. 넓은 AXI-Lite 데이터 경로 격리/삭제](#)
 - [육하원칙](#)
 - [Codex 프롬프트 - 삭제/격리](#)
 - [검증 프롬프트](#)
- [S01. DMA 기반 하드웨어 구조 작성](#)
 - [육하원칙](#)
 - [Codex 프롬프트 - DMA 구조 및 라우팅 복구](#)
 - [검증 프롬프트](#)
- [S02. bitstream/XSA 생성과 routing/timing](#)
 - [육하원칙](#)
 - [Codex 프롬프트](#)
 - [검증 프롬프트](#)
- [S03. PetaLinux/SD/serial 자동화](#)
 - [육하원칙](#)
 - [Codex 프롬프트](#)
 - [검증 프롬프트](#)
- [S04. 보드 register/DMA smoke](#)
 - [Codex 프롬프트](#)
 - [검증 프롬프트](#)
- [S05. C gemv_hw_test로 fake gemv FPGA PASS](#)
 - [Codex 프롬프트](#)
 - [검증 프롬프트](#)
- [S06. Runtime 모든 GEMV FPGA backend화](#)
 - [Codex 프롬프트](#)
 - [검증 프롬프트](#)
- [S07. HDMI console 최종 demo](#)
 - [Codex 프롬프트](#)
 - [검증 프롬프트](#)
- [S08. Freeze와 발표 리허설](#)
 - [4. 팀원에게 넘길 것](#)
 - [5. 최종 판정표](#)
 - [6. 절대 금지 문구](#)

SmolLM2 Zybo 5일 DMA/Agent 선형 실행 가이드 v4

목적: 현 프로젝트 상태에서 넓은 AXI-Lite 데이터 경로를 active build에서 제거하고, **AXI DMA + AXI-Stream GEMV + Linux C runtime + HDMI console demo**까지 선형으로 진행하기 위한 작업 문서다.

대상: 사용자, 팀원 A(runtime/Linux), 팀원 B(Vivado/PetaLinux), Codex/AI agent.

최종 목표: Zybo Z7-20 Linux console에서 smolLM2_chat --backend fpga --require-fpga가 실행되고, Transformer 내부 Q8_0 GEMV가 전부 FPGA backend를 탄다.

0. 지금 프로젝트 판정

현재 업로드 프로젝트 기준 판정은 다음과이다.

살릴 것:

```
vivado_ip/rtl/gemv_q8_0_stream_core.v
vivado_ip/tb/tb_gemv_q8_0_stream_core.sv
scripts/run_gemv_sim.tcl
```

```
golden/fake_gemv/*
fpga_layout/q8_0_lane16/*
runtime_c/*
```

버릴 것 또는 active build에서 제거할 것:

- AXI-Lite STREAM_DATA 반복 write로 weight/scale을 보내는 구조
- AXI-Lite INPUT_DATA 반복 write로 input vector를 보내는 구조
- AXI-Lite RESULT_DATA 반복 read로 output vector를 읽는 구조
- DMA 없는 create_zybo_gemv_hw.tcl 최종 경로
- smoke-only bitstream을 GEMV 성공물로 취급하는 것

현재 bitstream:

- version/status register smoke용이다.
- full GEMV datapath와 DMA path가 없다.

현재 최종 구조 적합성:

- 부적합. DMA 기반으로 갈아엎어야 한다.

프로젝트 목표는 낮추지 않는다.

금지:

- q_proj 하나만 FPGA로 보내고 성공 처리
- fake_gemv 전용 하드코딩 IP
- scale 적용 제거
- mode=0/mode=1 제거
- 16-lane을 임의로 박살내기
- CPU fallback을 최종 성공으로 처리
- AXI-Lite로 weight 전체를 반복 전송

1. 최종 하드웨어/소프트웨어 흐름

PC/Vivado/Codex

- > GEMV stream core 유지
- > AXI DMA + AXIS FIFO + GEMV DMA top 구성
- > bitstream 생성
- > XSA export

Petalinux/boot packaging/Codex

- > XSA 반영
- > device tree/DMA/UIO 또는 /dev/mem 접근 준비
- > BOOT.BIN/image.ub/SD 구성

Zybo board/Codex + 사용자

- > 사용자는 SD 삽입/전원/부팅만 수행
- > Codex는 serial terminal로 보드 조작
- > C gemv_hw_test 실행
- > fake_gemv FPGA PASS

Runtime/Codex

- > SmoLLM2-135M Q8_0 GGUF load
- > 모든 Q8_0 GEMV call을 gemv_backend_run()으로 통과
- > backend=fpga, require-fpga 모드
- > HDMI + USB keyboard console에서 demo

최종 demo 명령:

```
./smollm2_chat \  
--model /mnt/sd/SmollM2-135M-Instruct-Q8_0.gguf \  
--backend fpga \  
--require-fpga \  
--max-new-tokens 16
```

최종 출력에 반드시 있어야 하는 숫자:

```
total_gemv_calls: N  
fpga_gemv_calls: N  
cpu_gemv_fallbacks: 0
```

2. Codex/AI agent 권한과 안전 규칙

이번 프로젝트는 AI agent에 적극 위임한다. Codex는 다음을 수행할 수 있다고 가정한다.

- 허용:
- 파일 삭제/이동/격리
 - apt/pip/cmake 등 필요한 패키지 설치
 - Vivado/Vitis/PetaLinux 실행
 - PetaLinux가 없으면 설치 경로 탐색, settings.sh source, 설치 안내/자동화
 - bootgen 또는 Vitis packaging 사용
 - SD 카드 mount/remount/write
 - BOOT.BIN/image.ub/rootfs에 파일 복사
 - USB serial terminal 접속
 - 보드 Linux shell 명령 실행
 - 데모 앱 build/run

단 하나의 예외:

SD 카드 전체 포맷, 파티션 삭제, dd 쓰기처럼 host disk를 날릴 수 있는 destructive 작업은 반드시 lsblk 결과를 보여주고 사용자가 SD device path를 1회 확인한 뒤 수행한다.

장시간 실행 규칙:

Vivado/PetaLinux/synthesis/implementation/bitstream/buildroot/rootfs build는 장시간 작업이다. Codex는 1-2분마다 tail을 보며 컨텍스트를 태우지 않는다. 로그는 파일로 redirect하고, 최소 10분 단위 또는 프로세스 종료 시점에만 확인한다. 보고는 마지막 80줄과 ERROR/CRITICAL WARNING/FAIL grep 요약만 한다.

3. 선행 실행표

단 계	이름	주 담당	입력	산출물	통과 기준
S00	낡은 파일 격리/삭제	사용자 +Codex	현재 repo	deprecated 폴더, active path 정리	AXI-Lite data path가 active build에서 빠짐
S01	DMA 하드웨어 재설계	팀원 B+Codex	stream core, fake_gemv	DMA top RTL, BD Tcl	AXI DMA MM2S/S2MM 경로 존재
S02	라우팅/timing 복구	팀원 B+Codex	DMA design	bitstream, XSA	full GEMV datapath 포함 bitstream 생성
S03	PetaLinux/SD/boot 구성		XSA/bit	SD boot files	보드 Linux가 새 HW로 부팅

단계	이름	주 담당	입력	산출물	통과 기준
		팀원 B+Codex			
S04	보드 serial/register/DMA smoke	사용자 +Codex	booted board	register/DMA log	version/status, DMA 접근 확인
S05	C gemv_hw_test	팀원 A+Codex	DMA HW info, golden	gemv_hw_test	fake_gemv FPGA PASS
S06	runtime 모든 GEMV backend화	팀원 A+Codex	GGUF/layout/HW driver	smollm2_chat	total_gemv == fpga_gemv
S07	HDMI console 최종 demo	사용자 +Codex	SD/board/app	demo log/video	대화 출력 + fallback 0
S08	freeze	전원	final artifacts	release folder	재부팅 2회 재현

팀원에게 넘기는 것은 긴 설명이 아니라 해당 단계 파일 묶음 + 해당 단계 Codex 프롬프트다.

S00. 낡은 AXI-Lite 데이터 경로 격리/삭제

육하원칙

왜:
현재 full wrapper가 AXI-Lite로 weight/input/output을 운반해서 최종 구조로 부적합하다.

누가:
사용자 또는 Codex.

언제:
지금 즉시. DMA 구조를 새로 만들기 전.

어디서:
/home/user22/Desktop/smollm2-zybo

무엇을:
낡은 AXI-Lite data-path 파일을 active build에서 제거하고 deprecated로 격리한다.

어떻게:
아래 삭제/격리 프롬프트를 Codex에 실행시킨다.

확인:
active scripts에서 INPUT_DATA/STREAM_DATA/RESULT_DATA 기반 최종 경로가 사라져야 한다.

Codex 프롬프트 - 삭제/격리

현재 프로젝트에서 낡은 AXI-Lite data-path 구조를 active build에서 제거하라.

중요:
완전 삭제 전에 deprecated/old_axi_lite_bringup/ 아래로 백업 이동한다.
로그와 증거는 남긴다.
계산 core인 vivado_ip/rtl/gemv_q8_0_stream_core.v는 절대 삭제하지 마라.
RTL 시뮬레이션에 필요한 testbench와 golden도 삭제하지 마라.

문제 구조:

- AXI-Lite INPUT_DATA 반복 write로 input vector 전송
- AXI-Lite STREAM_DATA 반복 write로 weight/scale stream 전송
- AXI-Lite RESULT_DATA 반복 read로 output vector 전송
- DMA 없는 create_zybo_gemv_hw.tcl / create_zybo_gemv_smoke_hw.tcl을 최종 구조로 사용하는 것
- smoke bitstream을 full GEMV bitstream으로 취급하는 것

보존할 파일:

- vivado_ip/rtl/gemv_q8_0_stream_core.v
- vivado_ip/tb/tb_gemv_q8_0_stream_core.sv
- vivado_ip/tb/tb_gemv_q8_0_stream_core.v, 있으면 보존
- scripts/run_gemv_sim.tcl
- golden/fake_gemv/*
- fpga_layout/q8_0_lane16/*
- runtime_c/*
- logs/gemv_sim_result.txt

active build에서 제거하거나 deprecated로 이동할 파일:

- vivado_ip/rtl/gemv_q8_0_axi_lite.v
- vivado_ip/rtl/gemv_q8_0_axi_lite_smoke.v
- scripts/create_zybo_gemv_hw.tcl
- scripts/create_zybo_gemv_smoke_hw.tcl

수정할 문서:

- docs/interface_contract.md 맨 위에 DEPRECATED 문구를 추가하라.
이유: AXI-Lite INPUT_DATA/STREAM_DATA/RESULT_DATA 경로는 최종 구조가 아니다.
- docs/rtl_next_step.md 맨 위에 DEPRECATED 문구를 추가하라.
이유: block_acc-only/CPU-scale 정책이 남아 있으면 현재 fixed-scale FPGA 정책과 충돌한다.

새로 만들 파일:

- docs/deprecated_axi_lite_data_path.md
- logs/s00_deprecate_axi_lite_path.txt

검증:

1. active scripts 디렉터리에 create_zybo_gemv_dma_hw.tcl이 아직 없으면 TODO로 표시한다.
2. grep -Rni "REG_STREAM_DATA\|REG_INPUT_DATA\|REG_RESULT_DATA\|STREAM_DATA\|INPUT_DATA\|RESULT_DATA" vivado_ip scripts docs 결과를 저장한다.
3. 위 키워드가 deprecated 폴더와 deprecated 문서에만 남아야 한다.
4. active build 문서에는 이 경로를 사용하지 말라는 문구가 있어야 한다.

금지:

- stream core 삭제 금지.
- fake_gemv golden 삭제 금지.
- 기존 로그 삭제 금지.
- 기능을 줄여 성공 처리 금지.

검증 프롬프트

S00 검증만 수행하라.

새 RTL을 작성하지 마라.

다음은 확인하라.

1. vivado_ip/rtl/gemv_q8_0_stream_core.v가 남아 있는가?
2. deprecated/old_axi_lite_bringup/ 아래에 기존 AXI-Lite wrapper와 smoke 파일이 이동되었는가?
3. active build에서 create_zybo_gemv_hw.tcl, create_zybo_gemv_smoke_hw.tcl이 최종 경로로 사용되지 않는가?
4. docs/interface_contract.md, docs/rtl_next_step.md 상단에 DEPRECATED 경고가 있는가?
5. STREAM_DATA/RESULT_DATA/INPUT_DATA 키워드가 active 최종 경로에 남아 있지 않은가?

결과를 logs/s00_verify_deprecated_path.txt에 작성하라.

S01. DMA 기반 하드웨어 구조 작성

육하원칙

왜:

AXI-Lite는 제어용이고 대용량 weight/scale/input/output 경로가 아니다.

누가:

팀원 B + Codex.

언제:

S00에서 낡은 경로를 active build에서 제거한 직후.

어디서:

Vivado 2024.2.2 사용 PC.

무엇을:

AXI DMA + AXIS FIFO + GEMV stream core + control AXI-Lite wrapper.

어떻게:

아래 강화 프롬프트를 Codex에 넣는다.

확인:

block design에 AXI DMA MM2S -> FIFO -> GEMV s_axis와 GEMV m_axis -> FIFO -> S2MM 또는 BRAM output이 있어야 한다.

Codex 프롬프트 - DMA 구조 및 라우팅 복구

현재 Zybo Z7-20 / Vivado 2024.2.2 프로젝트의 Q8_0 GEMV 하드웨어 구조를 복구한다.

현재 확인된 문제:

1. 현재 block design에는 AXI DMA가 없다.
2. scripts/create_zybo_gemv_hw.tcl과 scripts/create_zybo_gemv_smoke_hw.tcl은 PS7 + AXI interconnect + GEMV AXI-Lite 모듈만 생성한다.
3. M_AXIS_MM2S -> GEMV stream input 경로가 없다.
4. S2MM 또는 output DMA 경로가 없다.
5. gemv_q8_0_axi_lite.v는 INPUT_DATA, STREAM_DATA, RESULT_DATA를 통해 대용량 데이터를 AXI-Lite로 전송한다.
6. 현재 smoke bitstream은 version/status register 접근용일 뿐 full GEMV datapath도 DMA path도 포함하지 않는다.
7. full GEMV AXI-Lite wrapper는 synthesis는 됐지만 route/timing 실패했다.
8. docs/rtl_next_step.md에는 예전 block_acc-only/CPU-scale 정책이 남아 있으므로 현재 정책과 충돌한다.

중요:

이 문제를 기능 축소로 해결하지 마라.

프로젝트 목표는 SmoLLM2 Transformer 내부 모든 Q8_0 GEMV를 FPGA backend로 offload하는 것이다.

따라서 q_proj 하나만 처리하거나, fake_gemv 전용 IP로 만들거나, mode=0/mode=1 중 하나를 제거하거나, scale 적용을 제거하거나, lane 수를 임의로 줄이는 것은 금지다.

보존할 핵심 파일:

- vivado_ip/rtl/gemv_q8_0_stream_core.v

이 파일은 s_axis_tdata/tvalid/tready/tlast 입력과 m_axis_tdata/tvalid/tready/tlast 출력을 가진 stream core이므로 계산 core로 유지한다.

폐기/대체할 구조:

- AXI-Lite STREAM_DATA로 weight/scale을 밀어 넣는 구조
- AXI-Lite INPUT_DATA로 대량 input을 밀어 넣는 구조
- AXI-Lite RESULT_DATA로 output vector 전체를 읽는 구조
- smoke bitstream을 최종 GEMV bitstream으로 취급하는 구조

새 목표 구조:

Zynq PS

-> M_AXI_GPO

-> AXI-Lite control/status registers

-> AXI DMA control registers

-> 필요 시 AXI BRAM Controller control/data window

DDR

- > PS S_AXI_HP0
 - > AXI DMA MM2S
 - > AXIS Data FIFO
 - > GEMV s_axis input

GEMV m_axis output

- > AXIS Data FIFO
- > AXI DMA S2MM
- > PS S_AXI_HP0
- > DDR output buffer

input_i16 vector 경로:

1차 권장:

- AXI BRAM Controller + input BRAM
- PS가 input vector를 BRAM address window에 burst/memcpy 방식으로 쓴다.

대체:

- 별도 AXIS input loader 또는 input packet header.

금지:

- 최종 구조에서 INPUT_DATA 단일 AXI-Lite register를 반복 write하는 방식.

AXI-Lite register map은 control/status 중심으로 축소한다:

- VERSION
- CONTROL
- STATUS
- ERROR_CODE
- MODE
- SCALE_SHIFT
- IN_FEATURES
- OUT_FEATURES
- INPUT_BASE 또는 INPUT_BRAM_BASE/selector
- WEIGHT_STREAM_LENGTH
- RESULT_LENGTH
- START
- DONE
- DEBUG_ROW
- DEBUG_BLOCK
- DEBUG_LANE

AXI-Lite에서 제거하거나 deprecated 처리할 것:

- INPUT_DATA
- STREAM_DATA
- STREAM_LAST
- RESULT_DATA
- RESULT_ROW
- RESULT_BLOCK
- RESULT_LANE
- RESULT_LAST

단, 디버그용으로 남기더라도 최종 데모 경로에서는 사용 금지라고 문서화한다.

Vivado block design 요구:

1. processing_system7_0 생성.
2. PCW_USE_M_AXI_GPO 활성화.
3. PCW_USE_S_AXI_HP0 활성화.
4. FCLK_CLK0는 우선 50 MHz로 설정한다.
5. proc_sys_reset을 사용한다. FCLK_RESET0_N을 IP reset에 직접 연결하지 않는다.
6. AXI DMA IP를 추가한다.

7. AXI DMA는 simple mode 우선, scatter-gather 비활성화 가능.
8. AXI DMA S_AXI_LITE는 PS M_AXI_GP0에 연결한다.
9. AXI DMA M_AXI_MM2S와 M_AXI_S2MM은 PS S_AXI_HP0 DDR 경로에 연결한다.
10. AXI DMA M_AXIS_MM2S -> axis_data_fifo -> GEMV s_axis로 연결한다.
11. GEMV m_axis -> axis_data_fifo -> AXI DMA S_AXIS_S2MM으로 연결한다.
12. GEMV control/status AXI-Lite wrapper는 PS M_AXI_GP0에 연결한다.
13. input vector BRAM을 쓴다면 AXI BRAM Controller를 PS M_AXI_GP0에 연결하고 GEMV core의 input read port와 dual-port BRAM으로 연결한다.
14. address map을 logs/hw_dma_address_map.txt에 저장한다.

Clock/timing 정책:

- 첫 route 성공 목표는 PL 50 MHz다.
- CPU/PS clock은 낮추지 않는다.
- GEMV core, AXI DMA, AXIS FIFO, AXI interconnect가 같은 FCLK_CLK0 또는 명확한 clock converter를 사용해야 한다.
- RTL의 X_INTERFACE_PARAMETER에 FREQ_HZ 100000000이 박혀 있으면 50 MHz 설정과 맞게 수정하거나 제거한다.
- 50 MHz 성공 후 75 MHz, 100 MHz 순서로 시도한다.

RTL timing 복구:

- gemv_q8_0_stream_core.v 기능은 유지한다.
- ST_BLOCK_DONE에서 16 lane의 block_acc * scale_q * round_shift * row_acc add를 한 사이클에 몰아넣지 말고 pipeline stage로 분리한다.
- 권장 stage:
 - Stage A: block_acc latch
 - Stage B: scale multiply
 - Stage C: rounding shift
 - Stage D: row_acc accumulation
- 16-lane MAC 기능은 유지한다.
- 필요하다면 scale stage만 lane-wise 또는 4-lane group time-mux 할 수 있다. 단 모든 GEMV는 여전히 FPGA에서 처리되어야 하며 CPU scale fallback은 금지다.
- 큰 buffer는 FF/LUT array가 아니라 BRAM/XPM memory로 강제한다.
- output vector 전체를 register array로 노출하지 않는다.
- 불필요한 keep/dont_touch/mark_debug를 제거한다.

반드시 만들 새 파일:

- vivado_ip/rtl/gemv_q8_0_dma_top.v
- vivado_ip/rtl/gemv_q8_0_ctrl_axi_lite.v
- scripts/create_zybo_gemv_dma_hw.tcl
- scripts/build_zybo_gemv_dma_bitstream.tcl
- scripts/report_failed_impl.tcl
- docs/hw_dma_architecture.md
- docs/hw_route_recovery.md
- docs/interface_contract_dma.md

수정 또는 deprecated 표시할 파일:

- docs/interface_contract.md
 - 기존 AXI-Lite INPUT_DATA/STREAM_DATA/RESULT_DATA 경로는 deprecated라고 표시한다.
- docs/rtl_next_step.md
 - 현재 정책과 충돌하므로 deprecated 문구를 맨 위에 추가한다.

report_failed_impl.tcl 요구:

실패한 implementation이 있으면 다음 report를 생성한다.

- reports/full_gemv_util_hier.rpt
- reports/full_gemv_timing_summary.rpt
- reports/full_gemv_route_status.rpt
- reports/full_gemv_congestion.rpt
- reports/full_gemv_qor_suggestions.rpt

장시간 Vivado 실행 규칙:

- Vivado/PetaLinux 명령은 로그를 파일로 redirect한다.
- 실행 중 진행상황을 1-2분마다 반복 확인하지 마라.
- 긴 실행을 시작하면 최소 10분 단위 또는 프로세스 종료 시점에만 확인한다.
- 중간 polling을 하더라도 전체 로그를 붙여넣지 말고 마지막 80줄만 본다.

- 최종 보고에는 전체 로그를 붙여넣지 말고, grep으로 ERROR/CRITICAL WARNING/FAIL만 요약한다.
- full log는 logs/*.txt 파일로 남긴다.
- 성공/실패 판단에 필요한 요약만 docs/hw_dma_bringup_result.md에 쓴다.

장시간 실행용 명령 예:

```
nohup /opt/Xilinx/Vivado/2024.2/bin/vivado -mode batch -source scripts/build_zybo_gemv_dma_bitstream.tcl > logs/vivado_dma_build_console.log 2>&1 & echo $! > logs/vivado_dma_build.pid
```

진행 확인은 다음만 사용:

```
ps -p $(cat logs/vivado_dma_build.pid)
tail -80 logs/vivado_dma_build_console.log
grep -Rni "ERROR\\|CRITICAL WARNING\\|Timing constraints are not met\\|write_bitstream completed\\|failed" logs hw 2>/dev/null | tail -100
```

성공 기준:

1. full GEMV datapath가 포함된 bitstream이 생성된다.
2. AXI DMA MM2S -> GEMV s_axis 경로가 block design에 존재한다.
3. GEMV m_axis -> AXI DMA S2MM 또는 BRAM output path가 존재한다.
4. AXI-Lite는 control/status 용도로만 사용된다.
5. STREAM_DATA/RESULT_DATA 반복 MMIO 방식이 최종 경로에서 제거된다.
6. XSA가 export된다.
7. docs/hw_dma_bringup_result.md에 bitstream path, xsa path, GEMV base address, DMA base address, input/output buffer 방식이 기록된다.
8. 기능을 줄여 성공 처리하지 않는다.

실패 시 보고:

- WNS/TNS
- route congestion 위치
- hierarchical utilization 상위 10개 module
- 가장 긴 timing path 5개
- DMA BD 연결 상태
- AXI-Lite 데이터 경로가 남아 있는지 여부
- 다음 수정 제안

검증 프롬프트

S01 검증만 수행하라.

다음을 확인하고 logs/s01_dma_arch_verify.txt에 남겨라.

1. create_zybo_gemv_dma_hw.tcl이 존재하는가?
2. AXI DMA IP가 BD에 추가되는가?
3. PS S_AXI_HP0가 활성화되는가?
4. DMA M_AXIS_MM2S -> FIFO -> GEMV s_axis 연결이 있는가?
5. GEMV m_axis -> FIFO -> DMA S2MM 또는 명시적 output BRAM 경로가 있는가?
6. AXI-Lite는 control/status만 담당하는가?
7. INPUT_DATA/STREAM_DATA/RESULT_DATA가 final datapath에 남아 있지 않은가?
8. PL clock은 50 MHz부터 시작하는가?
9. stream core 기능과 mode=0/mode=1이 유지되는가?

실패하면 고치지 말고 먼저 무엇이 빠졌는지 목록으로 보고하라.

S02. bitstream/XSA 생성과 routing/timing

육하원칙

왜:
실제 보드에 올릴 full GEMV DMA bitstream이 필요하다.

누가:
팀원 B + Codex.

언제:
S01에서 DMA block design이 작성된 뒤.

어디서:
Vivado 2024.2.2 PC.

무엇을:
full GEMV datapath 포함 bitstream과 XSA.

어떻게:
장시간 실행 규칙으로 batch build를 돌린다.

확인:
write_bitstream 성공, XSA export, timing report 저장.

Codex 프롬프트

S02 bitstream/XSA 생성을 수행하라.

전제:

- S01에서 create_zybo_gemv_dma_hw.tcl과 build_zybo_gemv_dma_bitstream.tcl이 생성되어 있다.
- 목표는 full GEMV DMA datapath 포함 bitstream이다.
- smoke-only bitstream은 성공으로 치지 않는다.

작업:

1. logs, reports, hw/zybo_gemv_dma 폴더를 준비한다.
2. Vivado batch build를 nohup으로 실행한다.
3. 진행상황을 너무 자주 확인하지 않는다.
4. 완료 후 bitstream과 XSA 경로를 찾는다.
5. timing/utilization/congestion report를 reports/에 저장한다.
6. docs/hw_dma_bringup_result.md를 작성한다.

실행 예:

```
mkdir -p logs reports
nohup /opt/Xilinx/Vivado/2024.2/bin/vivado \
  -mode batch \
  -source scripts/build_zybo_gemv_dma_bitstream.tcl \
  > logs/vivado_dma_build_console.log 2>&1 &
echo $! > logs/vivado_dma_build.pid
```

완료 후 요약:

```
tail -80 logs/vivado_dma_build_console.log
grep -Rni "ERROR\|CRITICAL WARNING\|FAIL\|Timing constraints are not met\|write_bitstream\|failed" logs hw reports 2>/dev/null | tail
-100
```

성공 기준:

- full GEMV datapath 포함 .bit 존재
- .xsa 존재
- AXI DMA base address와 GEMV ctrl base address 기록

- timing 실패 여부 명시
- smoke-only가 아님을 문서에 명시

검증 프롬프트

S02 산출물 검증만 수행하라.

새 기능 추가하지 마라.

확인:

1. 생성된 bitstream이 smoke-only가 아니라 DMA + full GEMV datapath를 포함하는가?
2. XSA가 export되었는가?
3. address map에 GEMV control, AXI DMA, input BRAM이 있으면 BRAM controller 주소가 있는가?
4. timing WNS/TNS가 기록되었는가?
5. write_bitstream 성공 로그가 있는가?
6. 실패했다면 report_failed_impl.tcl 결과가 생성되었는가?

결과를 logs/s02_bitstream_xsa_verify.txt에 작성하라.

S03. PetaLinux/SD/serial 자동화

육하원칙

왜:

bitstream/XSA를 보드 Linux에 반영해야 C 프로그램이 FPGA를 호출할 수 있다.

누가:

팀원 B + Codex. 사용자는 SD 삽입, 보드 전원, 필요 시 destructive SD 작업 승인만 한다.

언제:

S02 XSA가 나온 직후.

어디서:

PetaLinux/Vitis/bootgen 가능한 PC와 SD 카드가 연결된 PC.

무엇을:

BOOT.BIN, image.ub, rootfs 파일, demo app 복사.

어떻게:

Codex가 설치/경로/SD read-only 문제를 해결하며 진행한다.

확인:

SD bootfs/rootfs에 새 파일이 들어가고 보드가 부팅된다.

Codex 프롬프트

S03 PetaLinux/SD/serial 자동화를 수행하라.

목표:

S02에서 나온 XSA/bitstream을 반영해 Zybo Z7-20이 새 GEMV DMA hardware로 부팅되게 만든다.

사용자가 할 일은 SD 카드 삽입, 보드에 SD 장착, 전원 인가뿐이다.

Codex는 가능한 범위에서 설치, mount, copy, serial terminal 조작까지 수행한다.

권한:

- 필요한 패키지 설치 허용.
- PetaLinux/Vitis/Vivado settings.sh 탐색 및 source 허용.
- PetaLinux가 없으면 설치 가능 여부를 확인하고 설치 경로를 제안/수행한다.
- bootgen으로 FSBL + bitstream + U-Boot BOOT.BIN을 만드는 fallback 허용.
- SD card remount rw 허용.

- 파일 복사 허용.
- serial terminal 접속 허용.

안전:

- SD 전체 포맷, 파티션 삭제, dd write는 lsblk 결과를 보여주고 사용자가 SD device path를 1회 확인한 뒤 수행한다.
- host disk로 의심되는 장치에는 절대 쓰지 않는다.

작업 순서:

1. source 가능한 Vivado/Vitis/PetaLinux settings.sh를 찾는다.
2. petalinux-* 명령 존재 여부를 확인한다.
3. PetaLinux 가능하면 XSA를 반영해 project/config/build/package를 진행한다.
4. PetaLinux가 불가능하면 Vitis/bootgen fallback으로 BOOT.BIN 생성 가능 여부를 확인한다.
5. SD mount 상태를 확인한다.
6. read-only면 원인을 확인하고 remount rw 또는 fsck/권한 문제를 해결한다.
7. bootfs에 BOOT.BIN, image.ub 등 필요한 파일을 복사한다.
8. rootfs 또는 bootfs에 다음 파일을 복사한다.
 - smollm2_chat
 - gemv_hw_test
 - SmolLM2-135M-Instruct-Q8_0.gguf 또는 경로 안내
 - fpga_layout/q8_0_lane16/*
 - golden/fake_gemv/*
9. 사용자가 보드를 부팅하면 USB serial terminal로 접속한다.
10. uname, dmesg, ls /dev, devmem 가능 여부를 확인한다.
11. logs/s03_board_boot_log.txt에 부팅/serial 로그를 저장한다.

산출물:

- docs/sd_boot_packaging_result.md
- logs/s03_petalinux_or_bootgen_log.txt
- logs/s03_sd_copy_log.txt
- logs/s03_board_boot_log.txt

성공 기준:

- 보드 Linux가 새 bitstream/boot files로 부팅한다.
- serial shell 접근 가능하다.
- GEMV/DMA base address 확인 준비가 되어 있다.

검증 프롬프트

S03 검증만 수행하라.

확인:

1. 어떤 방식으로 boot image를 만들었는가? PetaLinux인가 bootgen fallback인가?
2. SD bootfs/rootfs에 무엇을 복사했는가?
3. read-only 문제는 해결되었는가?
4. serial terminal로 보드 shell에 접근했는가?
5. devmem 또는 대체 MMIO 접근 도구가 있는가?
6. dmesg에 DMA/UIO 관련 오류가 있는가?

결과를 docs/s03_boot_verify.md에 작성하라.

S04. 보드 register/DMA smoke

Codex 프롬프트

S04 보드 register/DMA smoke test를 수행하라.

전제:

- 보드는 새 SD로 부팅되어 serial shell 접근 가능하다.
- S02/S03 문서에 GEMV control base address와 AXI DMA base address가 기록되어 있다.

작업:

1. 보드에서 root 권한 또는 devmem 접근 권한을 확인한다.
2. GEMV VERSION/STATUS register를 읽는다.
3. AXI DMA register base를 읽고 reset/status를 확인한다.
4. DMA loopback이 가능하면 loopback test를 먼저 한다.
5. DMA MM2S/S2MM가 dmesg 오류 없이 동작하는지 확인한다.
6. 결과를 logs/s04_board_register_dma_smoke.txt에 저장한다.

성공 기준:

- GEMV VERSION이 읽힌다.
- DMA register가 읽힌다.
- DMA reset/status 확인이 된다.
- bus error 또는 kernel oops가 없다.

금지:

- register 읽기만 하고 full GEMV 성공이라고 쓰지 마라.

검증 프롬프트

S04 검증만 수행하라.

새 앱을 작성하지 마라.

logs/s04_board_register_dma_smoke.txt를 확인하고 다음을 판정하라.

- GEMV register read: PASS/FAIL
- DMA register read: PASS/FAIL
- bus error/kernel oops: 있음/없음
- 다음 단계 S05로 갈 수 있는가?

결과를 docs/s04_smoke_verify.md에 작성하라.

S05. C gemv_hw_test로 fake_gemv FPGA PASS

Codex 프롬프트

S05 gemv_hw_test를 작성/수정하고 보드에서 실행하라.

목표:

보드 Linux C 프로그램이 AXI DMA를 통해 fake_gemv weight/scale/input을 FPGA GEMV IP에 보내고 output을 DDR로 받아 golden과 비교한다.

입력:

- golden/fake_gemv/*

- fpga_layout/q8_0_lane16/* 또는 fake_gemv packet
- docs/interface_contract_dma.md
- docs/hw_dma_bringup_result.md

작업:

1. linux_app 또는 runtime_c 아래에 gemv_hw_test C 프로그램을 둔다.
2. DMA MM2S buffer에 Q8_0 packet을 넣는다.
3. input vector는 설계된 input BRAM 또는 input buffer에 넣는다.
4. output buffer를 0xCD 패턴으로 초기화한다.
5. GEMV control register에 mode, in_features, out_features, scale_shift, lengths를 설정한다.
6. DMA MM2S/S2MM을 시작한다.
7. GEMV start를 건다.
8. timeout을 두고 done을 기다린다.
9. output cache invalidate가 필요하면 처리한다.
10. golden과 비교한다.
11. mode=0 scaled와 mode=1 block_acc를 모두 테스트한다.

출력 예:

[FPGA GEMV HW TEST]

case: fake_gemv

mode=0 scaled: PASS

mode=1 block_acc: PASS

성공 기준:

- FPGA가 실제 계산한다.
- AXI-Lite STREAM_DATA/RESULT_DATA 경로를 쓰지 않는다.
- DMA 또는 명시된 BRAM/output 경로를 쓴다.
- mode=0/mode=1 모두 PASS.

금지:

- fake_gemv 전용 RTL 하드코딩 금지.
- CPU reference만 실행하고 PASS 금지.
- AXI-Lite data register 반복 전송 금지.

검증 프롬프트

S05 검증을 수행하라.

확인:

1. gemv_hw_test가 AXI DMA 또는 명시된 BRAM 경로를 사용하는가?
2. AXI-Lite data register 반복 전송을 쓰지 않는가?
3. mode=0 scaled PASS가 있는가?
4. mode=1 block_acc PASS가 있는가?
5. timeout/error/fallback이 없는가?
6. CPU reference와 비교 로그가 있는가?

결과를 docs/s05_fake_gemv_hw_pass_verify.md에 작성하라.

S06. Runtime 모든 GEMV FPGA backend화

Codex 프롬프트

S06 SmoLLM2 runtime의 모든 Q8_0 GEMV를 FPGA backend로 연결하라.

목표:

CPU-only가 아니라 FPGA 가속 chat runtime이다.

Transformer 내부 모든 Q8_0 2D matrix-vector GEMV 호출은 `gemv_backend_run()`을 통과하고, `backend=fpga --require-fpga`에서는 전부 FPGA로 실행되어야 한다.

입력:

- SmolLM2-135M-Instruct-Q8_0.gguf
- fpga_layout/q8_0_lane16/*
- docs/interface_contract_dma.md
- `gemv_hw_test`에서 검증된 FPGA driver 코드
- `runtime_c/*`

작업:

1. `smolllm2_chat` 또는 equivalent C runtime을 만든다.
2. GGUF 모델을 로드한다.
3. `tokenizer/chat` template은 가능한 기존 runtime 또는 `llama.cpp` 기반을 사용한다.
4. 모든 Q8_0 2D tensor GEMV 호출이 `gemv_backend_run()`을 지나가게 한다.
5. embedding lookup은 GEMV가 아니므로 CPU 허용.
6. RMSNorm/RoPE/softmax/sampling/KV cache는 CPU 허용.
7. `lm_head`도 GEMV이므로 FPGA로 보내는 것을 목표로 한다.
8. `activation float32 -> int16` 변환은 `ACT_SHIFT=8`부터 시작한다.
9. FPGA `output_i32 -> float` 복원은 `output_float = output_i32 / (1 << ACT_SHIFT)`로 시작한다.
10. `--require-fpga`에서는 CPU GEMV fallback이 1번이라도 발생하면 즉시 실패한다.
11. 매 응답마다 `total_gemv_calls`, `fpga_gemv_calls`, `cpu_gemv_fallbacks`를 출력한다.

실행:

```
./smolllm2_chat \  
  --model /mnt/sd/SmolLM2-135M-Instruct-Q8_0.gguf \  
  --backend fpga \  
  --require-fpga \  
  --max-new-tokens 16
```

성공 기준:

- 대화가 출력된다.
- `total_gemv_calls > 0`
- `fpga_gemv_calls == total_gemv_calls`
- `cpu_gemv_fallbacks == 0`

금지:

- `q_proj` 하나만 offload하고 성공 금지.
- CPU-only 성공 금지.
- fallback 숨김 금지.
- 360M/Q4 금지.

검증 프롬프트

S06 검증을 수행하라.

확인:

1. 모든 Q8_0 GEMV call이 `gemv_backend_run()`을 통과하는가?
2. `--require-fpga` 모드가 있는가?
3. CPU GEMV fallback이 발생하면 즉시 실패하는가?
4. `total_gemv_calls`, `fpga_gemv_calls`, `cpu_gemv_fallbacks`가 출력되는가?
5. `fpga_gemv_calls == total_gemv_calls`인가?
6. fallback이 0인가?
7. 대화 출력이 실제로 생성되는가?

결과를 docs/s06_runtime_fpga_backend_verify.md에 작성하라.

S07. HDMI console 최종 demo

Codex 프롬프트

S07 최종 HDMI console demo를 준비하고 실행하라.

전제:

- 보드는 새 SD로 부팅된다.
- HDMI display와 USB keyboard가 연결되어 있다.
- serial terminal도 연결 가능하다.
- smollm2_chat과 model/layout/golden이 보드에 있다.

작업:

1. HDMI console에서 로그인 또는 자동 로그인 상태를 확인한다.
2. USB keyboard 입력을 확인한다.
3. serial terminal에서도 동시에 로그를 볼 수 있으면 유지한다.
4. smollm2_chat 실행 스크립트 run_demo.sh를 만든다.
5. max-new-tokens는 8 또는 16으로 고정한다.
6. demo prompt 2개를 준비한다.
7. 실행 결과를 logs/final_demo_run_1.txt, logs/final_demo_run_2.txt에 저장한다.
8. 재부팅 후 한 번 더 실행한다.

성공 기준:

- HDMI console에서 사용자가 문장을 입력한다.
- assistant 출력이 나온다.
- backend: fpga
- require_fpga: true
- fpga_gemv_calls == total_gemv_calls
- cpu_gemv_fallbacks == 0

검증 프롬프트

S07 검증을 수행하라.

확인:

1. HDMI console에서 실행되었는가?
2. USB keyboard로 입력했는가?
3. assistant 출력이 있는가?
4. FPGA backend counters가 출력되는가?
5. fallback이 0인가?
6. 재부팅 후 재현되었는가?

결과를 docs/s07_final_demo_verify.md에 작성하라.

S08. Freeze와 발표 리허설

새 기능 금지.

문서 수정은 실행 명령과 로그 위치만.

데모 SD를 복제하거나 최소한 BOOT.BIN/image.ub/rootfs/app/model을 백업한다.

체크리스트:

- [] bitstream/XSA path 기록
- [] SD boot files 백업
- [] smollm2_chat 실행 명령 고정
- [] gemv_hw_test 실행 명령 고정
- [] demo prompt 고정
- [] fallback 0 로그 확보
- [] serial terminal 로그 확보
- [] HDMI console 사진/영상 확보
- [] 재부팅 후 2회 재현

4. 팀원에게 넘길 것

팀원 B에게 넘길 것

파일:

전체 repo zip 또는 최소 hw_pack
vivado_ip/rtl/gemv_q8_0_stream_core.v
vivado_ip/tb/tb_gemv_q8_0_stream_core.sv
scripts/run_gemv_sim.tcl
golden/fake_gemv/*
fpga_layout/q8_0_lane16/*

프롬프트:

S00 삭제/격리 프롬프트
S01 DMA 구조 및 라우팅 복구 프롬프트
S02 bitstream/XSA 프롬프트
S03 Petalinux/SD/serial 프롬프트

팀원 A에게 넘길 것

파일:

전체 repo zip 또는 최소 runtime_pack
runtime_c/*
fpga_layout/q8_0_lane16/*
golden/fake_gemv/*
Smollm2-135M-Instruct-Q8_0.gguf 경로
S02/S03 결과로 나온 base address/register map/DMA map

프롬프트:

S05 gemv_hw_test 프롬프트
S06 runtime 모든 GEMV FPGA backend화 프롬프트
S07 demo 프롬프트

5. 최종 판정표

상태	판정
XSim fake_gemv PASS만 있음	아직 보드 데모 아님
smoke bitstream version/status만 읽힘	PS-PL 주소 접근만 확인
AXI-Lite STREAM_DATA로 weight 전송	최종 구조 부적합
DMA MM2S/S2MM 포함 bitstream 생성	하드웨어 구조 통과 후보
gemv_hw_test fake_gemv PASS	FPGA GEMV 보드 실행 성공
smollm2_chat backend=fpga fallback 0	최종 데모 성공

6. 절대 금지 문구

Codex가 아래처럼 말하면 다시 시킨다.

"일단 AXI-Lite로 weight를 써서 동작 확인"

"DMA는 나중에"

"q_proj 하나만 offload"

"fallback이 있지만 데모는 성공"

"smoke bitstream으로 보드 bring-up 성공"

"scale은 CPU에서 처리"

"mode=1 debug는 제거"

이번 프로젝트는 모든 Q8_0 GEMV를 FPGA backend로 보내는 DMA 기반 가속 데모다.